Universidad Americana (UAM)

Ingeniería en Sistemas

Programación de Aplicaciones de Escritorio

Evaluación 3 – Corte 2

Informe de la Tarea sumativa 3

Caja de ventas concurrente con control de hilos

Dato	Valor
Modalidad	Trabajo grupal
Grupo	3
Docente	MSc. Freddy Luis López Barrios
Fecha de las pruebas	29 de octubre de 2026
Fecha de entrega	29 de octubre de 2026
Repositorio	https://github.com/katemyme/Evaluacion-3-Corte-2

1. Identificación del caso e integrantes

Caso: Tarea sumativa 3 – Caja de ventas concurrente con control de hilos (sesión 3, indicadores IL3.1, IL3.2 e IL3.3). **Módulo:** Caja_de_ventas. **Responsable principal del caso:** Moisés Abraham Valle Alonso.

Tabla 1. Integrantes del grupo 3

Integrante	Perfil de GitHub	Responsabilidad principal
Víctor Mateo Alcocer López	@katemyme	Estructura del proyecto, configuración Maven, menú principal y correcciones de integración.
Moisés Abraham Valle Alonso	@Argonauta0	Caso 3, Caja de ventas concurrente: vista, controlador y modelos.
André Marcel Guido Castrillo	@Andre-Guido	Caso 1, Reserva concurrente de entradas: lógica, controlador, interfaz y estilos.
Erving Josué Miranda Ríos	@ErvingMiranda	Caso 2, Procesamiento de pedidos con cancelación; integración y pruebas.

2. Objetivo y descripción del funcionamiento

El objetivo es simular una caja de ventas con tres vendedores que trabajan al mismo tiempo. Cada vendedor realiza cinco ventas de C$100, por lo que una simulación completa termina con 15 ventas y C$1,500. La aplicación permite iniciar y cancelar, muestra el avance y protege los totales compartidos.

La ventana muestra el total de ventas, la recaudación, el estado de la simulación, una barra y un contador por vendedor y una bitácora con cada venta confirmada. Al terminar cada simulación, la bitácora y la consola muestran una verificación: ventas × C$100 = recaudación y la suma de ventas por vendedor.

3. Hilos, sincronización y cancelación

3.1 Vendedores concurrentes

Cada vendedor es un ProcesadorVentasTask, que extiende Task de JavaFX (una clase pensada para trabajo en segundo plano que permite cancelar y publicar progreso). Las tres tareas se ejecutan en un ExecutorService con tres hilos, uno por vendedor. En cada simulación se crean tareas y un ejecutor nuevos. Cada venta simula entre 600 y 1400 ms de trabajo con Thread.sleep().

3.2 Recurso compartido

La clase CajaCompartida guarda el total de ventas y la recaudación. registrarVenta() es synchronized e incrementa ambos valores en el mismo bloque, de modo que una venta queda registrada completa y nunca se observa un contador distinto de la recaudación. La lectura obtenerEstadoActual() también es synchronized y devuelve los dos valores leídos en el mismo momento.

```
public synchronized EstadoCaja registrarVenta(String nombreVendedor, int numeroVentaVendedor) {
    totalVentas++;
    recaudacionTotal += MONTO_VENTA;
    ...
}
```

3.3 Cancelación y terminación

Cancelar llama a cancel(true) en las tres tareas, lo que interrumpe la espera de cada vendedor. La venta que estaba en curso se termina y se registra completa, como permite la guía, y luego el ciclo se detiene. Como Task marca la tarea como cancelada antes de que su hilo termine, la terminación real se confirma con un hilo supervisor que llama a awaitTermination() sobre el ejecutor. Después, con Platform.runLater(), se muestra el resultado y se vuelve a habilitar Iniciar. El hilo de JavaFX no espera en ningún momento.

3.4 Interfaz, inicio duplicado y cierre

Las barras y el estado de cada vendedor están enlazados con el progreso y el mensaje de su Task, y los totales se actualizan con Platform.runLater(), por lo que los vendedores nunca modifican controles directamente. Durante una simulación Iniciar está deshabilitado y el controlador también lo comprueba. Si se cierra la ventana con vendedores activos, el cierre se consume, se cancelan las tareas y la ventana se cierra cuando los tres hilos terminaron.

4. Resultados esperados y obtenidos

Tabla 2. Resultados registrados en la bitácora y en la consola

Simulación	Ventas	Recaudación	Suma por vendedor	Verificación	Estado
1	15	C$1,500	15	CONSISTENTE	Simulación completada
2	15	C$1,500	15	CONSISTENTE	Simulación completada
3	15	C$1,500	15	CONSISTENTE	Simulación completada
4	15	C$1,500	15	CONSISTENTE	Simulación completada
5	15	C$1,500	15	CONSISTENTE	Simulación completada
6 (cancelada)	11 (3 + 4 + 4)	C$1,100	11	CONSISTENTE	Cancelada y estabilizada
7 (reinicio tras cancelar)	15	C$1,500	15	CONSISTENTE	Simulación completada
8 (cierre de ventana)	9	C$900	9	CONSISTENTE	Ventana cerrada con 3/3 trabajadores terminados

Tabla 3. Pruebas de aceptación del caso 3

Prueba	Resultado esperado	Resultado obtenido	Estado
Cinco ejecuciones completas	15 ventas y C$1,500 en cada una	Simulaciones 1 a 5: 15 ventas y C$1,500, verificación CONSISTENTE	Cumple
Cancelación	Solo ventas confirmadas; recaudación = ventas × C$100; contador estable	11 ventas y C$1,100; el valor no cambió en los 4 s siguientes	Cumple
Inicio duplicado	No duplicar la simulación	Iniciar deshabilitado durante la ejecución; el clic adicional no tuvo efecto	Cumple
Cancelar y volver a iniciar	Tareas nuevas y datos restablecidos	La simulación 7 empezó desde 0 / 15 y terminó con 15 y C$1,500	Cumple
Cerrar con los tres vendedores activos	Cerrar sin dejar trabajadores activos	Consola: «Ventana cerrada con 3/3 trabajadores terminados (executor terminado: true)»; proceso terminado con código 0	Cumple

5. Pruebas realizadas y capturas

Las capturas corresponden a ejecuciones reales de la aplicación realizadas el 29 de octubre de 2026. La salida de consola de la sesión se conserva en informes/evidencias/caso3-salida-consola.txt.

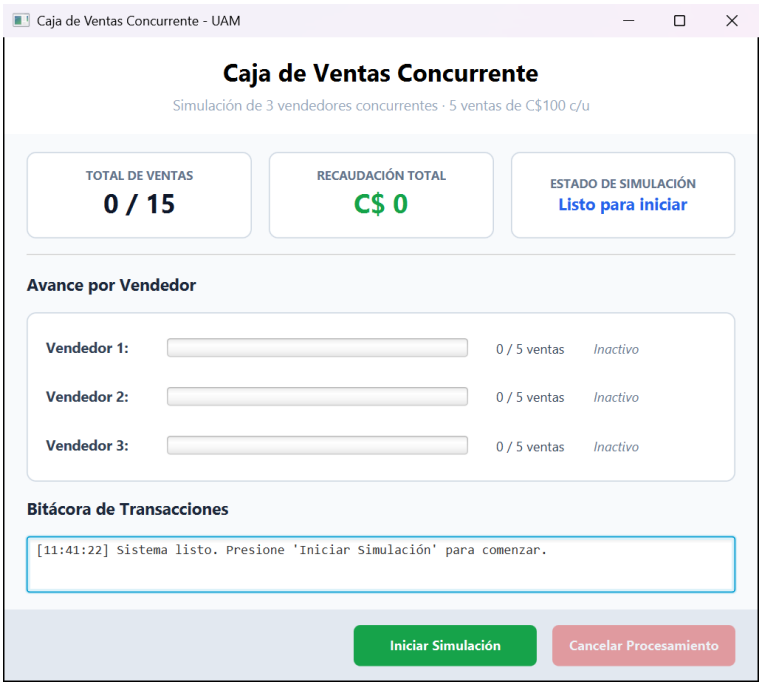


Figura 1. *Estado inicial: 0 / 15, C$0 y vendedores inactivos.*



Figura 2. *Simulación en curso: avance por vendedor, 7 ventas y C$700; Iniciar deshabilitado.*



Figura 3. *Ejecución completa 1: 15 / 15, C$1,500 y verificación CONSISTENTE.*



Figura 4. *Ejecución completa 2: 15 / 15, C$1,500 y verificación CONSISTENTE.*



Figura 5. *Ejecución completa 3: 15 / 15, C$1,500 y verificación CONSISTENTE.*



Figura 6. *Ejecución completa 4: 15 / 15, C$1,500 y verificación CONSISTENTE.*



Figura 7. *Ejecución completa 5: 15 / 15, C$1,500 y verificación CONSISTENTE.*



Figura 8. *Cancelación: 11 ventas, C$1,100 y estado «Cancelada y estabilizada».*



Figura 9. *Nueva simulación después de cancelar: los datos se restablecieron y comenzó desde cero.*

6. Distribución de responsabilidades

La primera tabla resume los aportes que pueden comprobarse en el historial de Git del repositorio. La segunda distingue al responsable principal de este caso de las contribuciones generales al proyecto.

Tabla 4. Aportes registrados en Git

Autor en Git	Aportes registrados en el repositorio
Víctor Mateo Alcocer López (ValcocerLopez)	Estructura inicial del monorepo, menú principal, corrección de los pom.xml y del controlador de la caja; fusión de la solicitud #1 (Reserva_Concurrente).
André Marcel Guido Castrillo (Andre)	Caso 1: vista reserva-view.fxml, hojas de estilo, clases Inventario, AgenteReserva y ReservaController; fusión de la solicitud #2 (caja_pedidos).
Moisés Abraham Valle Alonso (Argonauta0)	Caso 3: vista caja-ventas-view.fxml, CajaVentasController, CajaVentasApplication, modelos CajaCompartida y ProcesadorVentasTask, Launcher y comentarios.
Erving Josué Miranda Ríos (Erving Miranda)	Caso 2: PedidosController, ProcesadorPedidos, PedidosApplication, vista pedidos-view.fxml y pruebas JUnit; integración con master, revisión final del conjunto, README e informes.

Tabla 5. Distribución de responsabilidades en este caso

Integrante	Rol	Actividades
Moisés Abraham Valle Alonso	Responsable principal del caso	Vista caja-ventas-view.fxml, CajaVentasController, CajaVentasApplication, modelos CajaCompartida y ProcesadorVentasTask y Launcher.
Víctor Mateo Alcocer López	Contribución general al proyecto	Estructura del proyecto, configuración Maven y menú principal; correcciones de integración, incluidas las del controlador de la caja.
André Marcel Guido Castrillo	Contribución general al proyecto	Desarrollo del caso 1, Reserva concurrente de entradas.
Erving Josué Miranda Ríos	Contribución general al proyecto	Desarrollo del caso 2; integración, pruebas de los tres casos y documentación.

7. Conclusiones

La caja de ventas cumple los requisitos: los tres vendedores trabajan en segundo plano, cada venta se registra completa dentro de un método synchronized y el contador coincide siempre con la recaudación, incluso al cancelar. Las cinco ejecuciones terminaron con 15 ventas y C$1,500. La cancelación, el reinicio con tareas nuevas y el cierre de la ventana esperan la terminación real de los hilos sin bloquear la interfaz.

8. Repositorio y datos de ejecución

Para ejecutar desde IntelliJ IDEA: abrir la carpeta raíz del repositorio, seleccionar un SDK 21, elegir la configuración «Caso 3 - Caja de ventas» o «Menu principal» en la barra superior y pulsar Run. Las configuraciones se incluyen en la carpeta .run/ del repositorio.

Tabla 6. Datos de ejecución

Dato	Valor
Repositorio	https://github.com/katemyme/Evaluacion-3-Corte-2
Rama revisada	fix/revision-entrega (solicitud de integración #3 hacia master)
Módulo	Caja_de_ventas (clase de inicio ni.edu.uam.caja_de_ventas.Launcher)
JDK	Microsoft OpenJDK 21.0.13 (JDK 21)
JavaFX	21.0.6 (javafx-controls y javafx-fxml)
IDE	IntelliJ IDEA 2026.2.3
Sistema operativo	Windows 11 Pro
Configuración de IntelliJ	Caso 3 - Caja de ventas
Forma de ejecución en estas pruebas	Línea de comando equivalente a la configuración de IntelliJ (java -p <JavaFX 21.0.6 y target/classes> -m <módulo>/<Launcher>). El caso 1 se ejecutó directamente desde IntelliJ; para este caso se usó la misma línea porque el monitor donde estaba IntelliJ se desconectó durante la sesión de pruebas.